

ICS 214 Lab 4

Objectives:

1. Create a program that uses small functions to break down the complexity of the task.
2. Introduce passing by reference
3. Introduce reading input from a file.
4. Review character to integer conversion.

You'll notice that some of the functions you are required to write for this lab have parameters which include an ampersand in them to indicate they are being passed by reference (and some by const reference).

You should already be familiar with this from class, but if a reminder is helpful, here's a short recap of what that does, and how it differs from "pass by value":

Passing variables by reference: *

Passing by reference enables us to modify the original variable passed into a function. To pass a variable by reference, we simply declare the function parameter as a reference. This is done by adding an ampersand (&) in between the type and the variable.

```
1 void addOne(int& val) // val is passed by reference
2 {
3     val++;
4 }
```

In this example, `val` is simply a reference (alias) to the original variable. Changes made to the reference will affect the original argument passed in.

The following example shows this in action:

```
3 void addOne(int& val) // val is passed by reference
4 {
5     val++;
6 }
7
8 int main() {
9     int value{ 5 };
10
11     std::cout << "value: " << value << '\n';
12     addOne(value);
13     std::cout << "value: " << value << '\n';
14     return 0;
15 }
```

When we call `addOne(value)`, `ref` becomes a reference to `main`'s `value` variable. This snippet produces the output:

```
Microsoft Visual Studio Debug Console
value: 5
value: 6
```

As you can see, the function changed the argument from 5 to 6!

Passing variables by const reference:

A disadvantage of pass by value is that a copy of each argument is made and assigned to the function's parameters. For arguments larger than fundamental types (int, double, bool, char) this gets costly (more bytes need to be copied). References provide a way to avoid this penalty. Behind the scenes, references are implemented with pointers (4-8 bytes in size), so the cost of using a reference parameter is only 4-8 bytes. This allows us to pass larger objects more efficiently.

However, references allow the function to change the original argument. What if we don't want to allow the argument to change, but we want the performance benefit of pass by reference? The solution is to pass by const reference. This will cause the compiler to complain if there is an attempt to change the parameter.

Motivation for using const reference:

1. To ensure the value isn't changed by accident (by someone modifying code inside the function in the future).

```
36 void printMessage(const std::string& msg) // msg is a const reference
37 {
38     std::cout << msg << '\n';
39     //msg = "changed"; // compiler error: a const reference can't have its value changed!
40 }
```

2. To assure any clients calling our function that the argument they pass will not be modified.
3. To allow constants, or literals (in addition to variables) to be passed to the function.

```
36 void printMessage(const std::string& msg) // msg is a const reference
37 {
38     std::cout << msg << '\n';
39 }
40
41 int main()
42 {
43     std::string msg1 = "msg1";
44     printMessage(msg1);           // passing a var
45
46     const std::string msg2 = "msg2";
47     printMessage(msg2);           // passing a const
48
49     printMessage("msg3");         // passing a literal
50 }
```

Ok... let's get ready to use this information! The functions we'll be writing will take `std::string` objects as parameters (potentially much larger than 4-8 bytes). They don't need to be altered inside the functions, so we'll pass them by const reference. This will also allow us to pass string literals to our functions.

Credit Card Validation

Read through all of Step 1 before starting so that you get the “big picture” and don’t have to re-do anything. Take note of the hints (highlighted in yellow) so you don’t start in a more complicated/difficult direction.

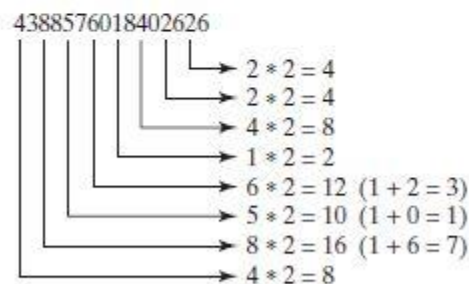
Step 1:

****6.37** (Financial: credit card number validation) Credit card numbers follow certain patterns. A credit card number must have between 13 and 16 digits. The number must start with the following:

- 4 for Visa cards
- 5 for MasterCard cards
- 37 for American Express cards
- 6 for Discover cards

In 1954, Hans Luhn of IBM proposed an algorithm for validating credit card numbers. The algorithm is useful to determine whether a card number is entered correctly or is scanned correctly by a scanner. Almost all credit card numbers are generated following this validity check, commonly known as the *Luhn check* or the *Mod 10 check*. It can be described as follows. (For illustration, consider the card number **4388576018402626**.)

1. Double every second digit from right to left. If doubling of a digit results in a two-digit number, add the two digits to get a single digit number.



2. Now add all single-digit numbers from Step 1.
 $4 + 4 + 8 + 2 + 3 + 1 + 7 + 8 = 37$
3. Add all digits in the odd places from right to left in the card number.
 $6 + 6 + 0 + 8 + 0 + 7 + 8 + 3 = 38$

4. Sum the results from Step 2 and Step 3.

$$37 + 38 = 75$$

5. If the result from Step 4 is divisible by **10**, the card number is valid; otherwise, it is invalid. For example, the number **4388576018402626** is invalid, but the number **4388576018410707** is valid.

Write a program that prompts the user to enter a credit card number as a string. Display whether the number is valid. Design your program to use the following functions:

Planning:

Now that you've seen the problem – take a moment to think about ways that you could break this problem down into smaller “bite-sized” tasks. Each task should have a singular purpose, specific input, and specific output. **Write a short list of these smaller tasks**, and then compare it to the function descriptions listed on the next page. I'm hoping the lists are fairly close. If you came up with less functions, consider which functions you didn't have listed, and if any of the functions you listed could be further subdivided into the ones on the next page.

We'll use the functions on the next page for the implementation, but an important part of the process is looking at the big picture, and then thinking about the chunks or individual tasks that you can use to break out functionality. When functions can be used to package smaller single purpose tasks, it generally makes your whole program easier to read/understand and makes it possible to re-use functionality. It will also make future modification easier. Chances are good that future development will only affect one task/function which means a lot of the code can be re-used (meaning less needs to change).

Function declarations for Credit.h:

```
// Given an int, return the sum of its individual digits
// hint: an easy way to parse an int into digits is by isolating
// the 'ones' column (via %10), and reducing the number (via /10)
// in a loop until it is 0.
// - params: int - a number to analyze (eg: 123)
// - return: an int representing the sum of digits (eg: 123 returns 6)
getSumOfDigits();

// Determine if a given card number starts with a given prefix
// Hint: research how the string.substr() function works.
// - param 1: a string representing the card number
// - param 2: a string representing the prefix
// - return: true if cardNumber starts with prefix (false otherwise)
startsWith();

// Determine if a cardNum has a valid prefix.
// - param 1: a string representing the cardNumber
// - return: true if cardNumber starts with string literals:
//           "4", "5", "6" or "37" (false otherwise)
// You could use string literals in this function, but it would
// much be cleaner to create a "Constants" namespace in your header
// file and create a const array of strings there (validPrefixes).
// that you can reference in this function.
// hint: you can make use of startsWith() in this function.
hasValidPrefix();

// Add up the odd-place digits from right to left
// - param 1: a string to represent the cardNumber
// - return: the sum of odd-place digits from right to left in the card
//           number (step 1, part 3).
// Eg: "34567" returns 15. (7+5+3)
sumOddDigitsRightToLeft();

// Add up the even-place digits from right to left
// - param 1: a string to represent the cardNumber
// - return: the sum of the (even-place digits * 2) from right to left
//           in the card number (step 1, part 1 & 2).
//           Special rule: multi digit #'s need to be converted by adding
//           their digits together before being added into the sequence.
// Eg: "34567" returns 11. ((6*2 = 12, converted to 1+2 = 3) + (4*2)) equals 11
// hint: make use of getSumOfDigits()
sumEvenDigitsRightToLeft();

// Determine if a card # is valid. Valid if:
//   - between 13 & 16 digits
//   - has a valid prefix
//   - sum of digits according to formula results in a number
//     evenly divisible by 10
// This is the "workhorse" function that co - ordinates
// all the others.
// - param 1: a string to represent the cardNumber
// - return: true if the card number is valid (false otherwise)
isCardValid();
```

Note: if you download this doc (rather than reading it within D2L), it should allow you to copy & paste the code into Visual Studio properly (so you don't have to re-type everything).

Getting started:

Very rarely does anyone write their program, hit compile to find and fix syntax errors, and then have everything work. A much better (and easier) approach is to tackle one function one at a time and test each one to ensure it works (before proceeding to the next).

Testing as you develop will give you the assurance that each function works properly. That way you know when you encounter an issue later on, it likely has to do with new code just wrote, rather than existing “somewhere in the program”. This “testing as you go” approach can save you a lot of time because it limits the scope of your errors to the function you are currently working on. This makes it much easier to figure out where things are going wrong.

If you get stuck on a function for longer than 30 minutes, please ask for help! I would much rather explain a concept than having you spinning your wheels or coming up with a method of making this work that is way more complicated than it needs to be!

- Create a new empty project called Lab4.
- Add the following source files: Main.cpp, Credit.cpp, Credit.h.
- Copy the forward declarations above into Credit.h. Work through Credit.h and add in the parameters you expect to use (taking into account how the parameters should be passed (value, reference, const reference). Copy the contents of Credit.h into Credit.cpp, and then comment everything in Credit.cpp out.
- #include “Credit.h” in both Main.cpp and Credit.cpp. Start from the top of Credit.cpp by uncommenting function definitions (one at a time) while implementing them and testing them. **Test each function by calling it from main() with a short string (“34567”) that is easy to calculate. Once that works, test on the sample card numbers (both invalid & valid) used in Step 1 above. Verify results match the illustration.**

Hints:

- **Don’t use any array-to-integer or string-to-integer functions.** Simply iterate from the back of the string to the front of the string (character by character), and convert each character to an int (similar to upper/lower case conversion) You can use `i=i-2` instead of `i--` to move through your for loop!
- You do not need to evaluate whether the total number of characters in the credit card string are even or odd. If you’re thinking this, you’re probably headed in the wrong direction.
- Both of the summation functions process from right to left. If the card number was 1234567, every other odd number from the right would be 7, 5, 3 1. Every even number would be 6, 4, 2.
- You will need to convert from a char to an int in both summation functions. You can do this easily by using math on character ascii values (as seen in class). Assume valid input.

Testing:

Efficiently completing this lab will likely require a combination of using the IDE debugger, and printing out values as you complete functions. Once you've verified your code functions as expected with the incorrect card number in the illustration test your code with the valid card numbers below. To test more invalid numbers simply change one of the digits. You may want to consider putting the credit card numbers in an array of strings. You can then loop through the array calling `isCardValid()` on each card number and printing out the result. Google: "test credit card numbers" if you want extra numbers to test with.

American Express: 378282246310005
Amexican Express: 371449635398431
Visa: 4012888888881881

Step 2:

Read credit card numbers in from a file. Use the `isCardValid()` function we wrote above to test them and output results.

We have read input from the keyboard before using `std::cin` (character input), but not from a file. Fortunately reading from an input file stream is very similar to reading from the keyboard. We just need to instantiate a file input stream and attempt to open a file first.

To start with, copy the `cards.txt` file from the `lab4` folder, and place it in the same folder as your `Main.cpp` file.

Include the `<fstream>` library.

Type the following code into your `main()` function. This will instantiate an input file stream variable and then use it to try to open a text file containing credit card numbers. Note that if we fail to open the file, we need to provide output to the user to let them know what happened. Test it to ensure the file is found.

```
std::string inputFilename = "cards.txt";
std::ifstream fin;
fin.open(inputFilename);
if (fin.is_open()) { ... }
else
{
    std::cout << "Error: could not open file: \"" << inputFilename << "\"" << std::endl;
    exit(1);
}
```

Once we have opened the file successfully, we can read from `fin` exactly the same way we read from `std::cin` (using the extraction operator `>>`). Let's set up a loop to read in credit card numbers until we are stopped from reading any further by the end of the file. When the end of

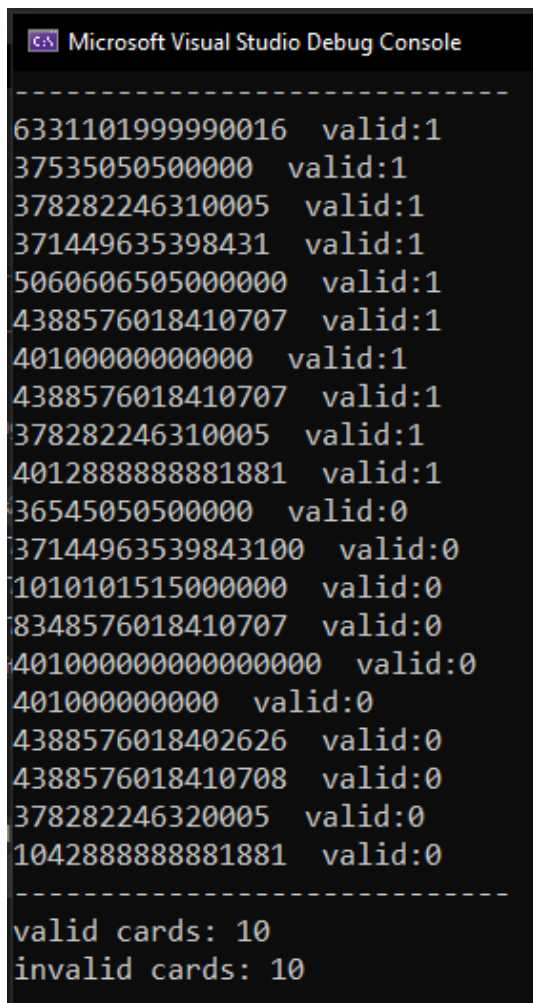
the file is hit, our while condition (below) will fail, and we need to close the file input stream to finish.

```
std::string ccNumber;  
while (fin >> ccNumber) { ... }  
fin.close();
```

For each iteration of the while loop we should have a new credit card number in our ccNumber variable. Print it out to the console to ensure everything is working so far.

Process each credit card number using the isCardValid() function you wrote in step 1, and provide output indicating if each card is valid. Outputting the Boolean result of the isCardValid() function will print out a 1 or 0. Keep track of the total count of valid/invalid cards and output them at the end.

Here is what the output should look like for your demo.



```
Microsoft Visual Studio Debug Console  
-----  
6331101999990016  valid:1  
37535050500000  valid:1  
378282246310005  valid:1  
371449635398431  valid:1  
5060606505000000  valid:1  
4388576018410707  valid:1  
4010000000000000  valid:1  
4388576018410707  valid:1  
378282246310005  valid:1  
4012888888881881  valid:1  
36545050500000  valid:0  
37144963539843100  valid:0  
1010101515000000  valid:0  
8348576018410707  valid:0  
401000000000000000  valid:0  
40100000000000  valid:0  
4388576018402626  valid:0  
4388576018410708  valid:0  
378282246320005  valid:0  
1042888888881881  valid:0  
-----  
valid cards: 10  
invalid cards: 10
```

Code Review

When you are finished this assignment, find a student who is also finished. Using the checklist (D2L | Extras | LabCodeReview), conduct a code review on each other's code **before** giving a demo of your assignment for marks.

Marking:

- Does it work as expected?
- Have parameters been passed by reference or const reference when appropriate?
- Variables: Have good variable & const names been used?
- Are variables in local scope instead of global? (except for the constants in Cards.h)
- Are there comments to explain anything non-trivial or not obvious?
- Have variables been initialized with best practices (uniform initialization).
- Have constants been used instead of "magic numbers" in the code (cardLengths & prefixes).
- Have header guards been added to .h files?

Task 1 [7/10]

Task 2 [3/10]

** Details on passing variables by reference copied from the following website (& modified slightly):*
<https://www.learncpp.com/cpp-tutorial/73-passing-arguments-by-reference/>